



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

مشروع مادة البرمجة المتوازية :

محرك المعالجة عالي الأداء لنظام التجارة الالكترونية

إعداد الطلاب

غيث محمد منذر تقي الدين

محمود محمد حسام الدين الخياط

زاهر قاسم السحلي

محمد ماهر عرابي

رامي عبد الحكيم عثمان

بإشراف: م. حذيفة عقيل

2026 - 2025

فهرس التقرير

1. ملخص المشروع وحدوده.
2. البنية البرمجية الفعلية ومسار تنفيذ عملية الشراء.
3. خريطة المتطلبات من 1 إلى 10.
4. المتطلب الأول: حماية البيانات المشتركة من التضارب.
5. المتطلب الثاني: إدارة الموارد والتحكم بعدد عمليات الشراء.
6. المتطلب الثالث: المعالجة غير المتزامنة.
7. المتطلب الرابع: المعالجة الدفعية لتقارير المبيعات.
8. المتطلب الخامس: توزيع الأحمال.
9. المتطلب السادس: التخزين المؤقت في أكثر من موضع.
10. المتطلب السابع: الأقفال الموزعة خارج قاعدة البيانات.
11. المتطلب الثامن: سلامة المعاملات.
12. المتطلب التاسع: محاكاة مئة مستخدم على العمليات الفعلية.
13. المتطلب العاشر: القياس وتحليل الاختناقات.
14. نتائج الاختبارات المحفوظة وكيفية عرض المشروع.
15. نقاط القوة، النواقص الصريحة، والخطوات الموصى بها.

ملخص المشروع وحدوده 1.

المشروع ليس واجهة متجر إلكتروني هدفها عرض عدد كبير من الصفحات، بل هو محرك خلفي يثبت قدرة منظومة التجارة الإلكترونية على العمل بصورة صحيحة تحت الضغط والتزامن. المحور الأساسي هو حماية المخزون والطلب والدفع من التضارب، وتنظيم عدد عمليات الشراء النشطة، ونقل الأعمال البطيئة إلى الخلفية، وتوزيع طلبات القراءة والكتابة على أكثر من نسخة للتطبيق، ثم إثبات ذلك بسكربتات اختبار ونتائج محفوظة.

التنفيذ الفعلي مبني كتطبيق واحد مقسم إلى وحدات مستقلة حسب المسؤولية؛ أي أنه ليس بنية خدمات مصغرة، لكنه يملك فصل مسؤوليات واضحاً داخل التطبيق نفسه. توجد وحدات للمنتجات والسلعة والطلبات والدفع والتقارير وقياس الأداء، بينما تُشغل البنية التحتية كحاويات منفصلة.

إطار العمل الخلفي: Django REST Framework

قاعدة البيانات: PostgreSQL

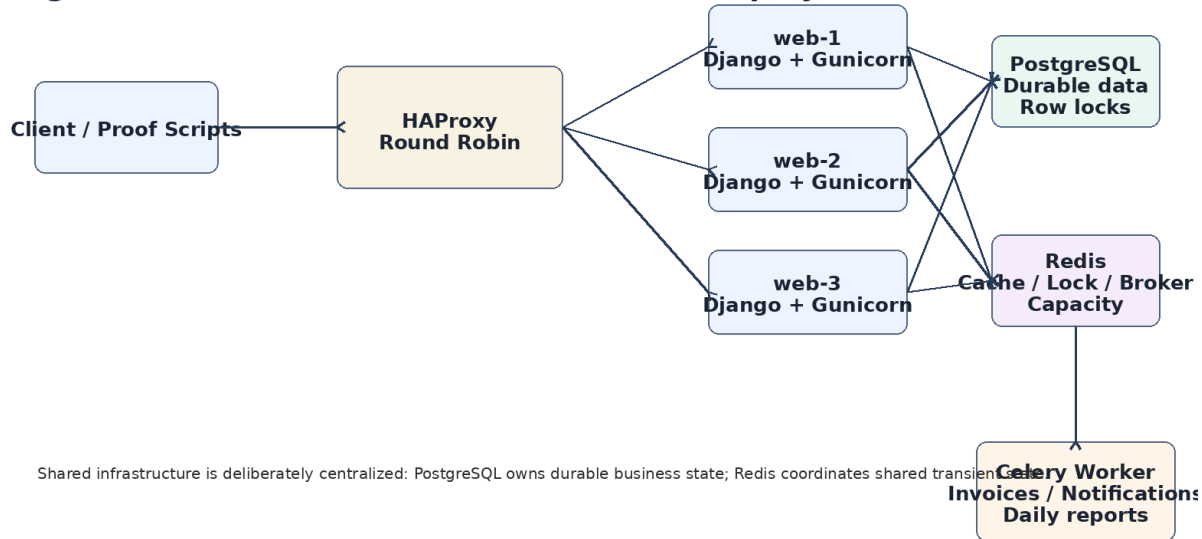
التخزين المؤقت والتنسيق المشترك: Redis

تنفيذ الأعمال الخلفية: Celery

موزع الأحمال: HAProxy

بيئة التشغيل: Docker Compose

High-Performance E-Commerce Backend - Deployment Architecture



الشكل 1: البنية العامة للنظام كما تم إعدادها للعرض والمراجعة

Checkout Flow - Critical Transactional Path



The business writes occur inside one PostgreSQL transaction. Redis coordinates admission and duplicate-checkout behavior outside the database.

الشكل 2: المسار التشغيلي لعملية إتمام الطلب من السلة حتى تنفيذ الأعمال الخلفية

البنية البرمجية الفعلية ومسار تنفيذ عملية الشراء 2.

أهم مسار في المشروع هو مسار إتمام الطلب يبدأ المستخدم بإضافة المنتج إلى السلة، ثم يرسل طلب الإتمام. قبل الدخول إلى المعاملة، يتم أولاً ضبط سعة عمليات الإتمام بواسطة عداد مشترك، ثم الحصول على قفل موزع خاص بالمستخدم لمنع التكرار، ثم تبدأ معاملة قاعدة البيانات التي تقفل السلة والعناصر والمنتجات ذات الصلة. بعد التحقق من الكمية، ينشئ النظام الطلب وتفاصيله وسجل الدفع، ويخفض المخزون ويفرغ السلة. ولا تُنفَّذ عملية إبطال التخزين المؤقت أو إرسال مهام الخلفية إلا بعد نجاح اعتماد المعاملة

المسار البرمجي	وظائفه الفعلية في المشروع	المكوّن
products/views.py	قراءة المنتجات وحماية إعادة بناء التخزين المؤقت	المنتجات
cart/models.py / cart/views.py	احتواء عناصر المستخدم قبل عملية الإتمام	السلة
orders/views.py	تنفيذ الإتمام وإنشاء الطلب وتفاصيله	الطلبات
payments/models.py	تسجيل دفعة ناجحة مرتبطة بطلب	الدفع
reports/tasks.py	إنشاء تقرير المبيعات اليومي على دفعات	التقارير
performance/	ضبط السعة، الأقفال الموزعة، وقياس زمن الطلبات	الأداء
docker-compose.yml / infra/haproxy/haproxy.cfg	تشغيل المكونات وتوزيع الطلبات	البنية التحتية

خريطة المتطلبات من 1 إلى 10 3.

دليل الإثبات	تنفيذ المشروع	المشكلة	رقم المتطلب
scripts/race_condition_test.py	أقفال صفوف ومعاملة واحدة	تضارب تعديل المخزون	1
scripts/resource_capacity_test.py	Redis محدد سعة مشترك في	استهلاك موارد الإتمام	2
scripts/async_queue_test.py	مهام خلفية بعد اعتماد المعاملة	بطء الاستجابة بسبب أعمال لاحقة	3
scripts/batch_processing_test.py	تجزئة التقرير اليومي إلى دفعات	معالجة مبيعات كثيرة دفعة واحدة	4

5	تحميل خادم واحد	توزيع دوري على ثلاث نسخ	scripts/load_distribution_test.py
6	قراءات متكررة من قاعدة البيانات	تخزين مؤقت للمنتجات والتقارير	scripts/cache_test.py
7	تنسيق عمليات تتجاوز قفل قاعدة البيانات	أقفال موزعة للسلة والتقارير وإعادة بناء التخزين المؤقت	scripts/distributed_lock_test.py
8	بقاء بيانات جزئية عند الفشل	معاملة شاملة مع عمليات لاحقة بعد الاعتماد	scripts/acid_transaction_test.py
9	إثبات الاستقرار على عمليات حقيقية	مئة مستخدم ينفذون مسار شراء كامل	scripts/stress_test_100_users.py
10	قياس الأداء وتحديد الاختناق	تسجيل زمن الطلبات وتحليل نتائج اختبار الضغط	performance/middleware.py + results/stress/

المتطلب الأول :حماية البيانات المشتركة من التضارب 4.

المشكلة المقصودة هنا هي أن مستخدمين أو أكثر قد يحاولون شراء المنتج نفسه في اللحظة نفسها. إذا قرأ كل طلب الكمية قبل أن يكتب الآخر التعديل، فقد تتجاوز الكمية المباعة المخزون الحقيقي. هذا يسمى سباقاً على البيانات، ولا يحل بمجرد فحص الكمية قبل الحفظ، لأن الفحص نفسه قد يحدث في أكثر من طلب بالتزامن.

في مشروعنا، يبدأ التنفيذ داخل معاملة قاعدة البيانات ثم يقفل صف السلة وعناصرها، وبعد ذلك يقفل صفوف المنتجات المطلوبة بترتيب معرفات ثابت. الترتيب الثابت يقلل احتمال التعارض الدائري عندما تحتوي سلال مختلفة على أكثر من منتج. بعد الحصول على القفل فقط يتم التحقق من المخزون ثم تخفيضه.

السيناريو العملي في المشروع

تم إعداد سكربت اختبار بكمية ابتدائية تساوي خمس وحدات وعشرين مستخدماً متزامناً. النتيجة المحفوظة أثبتت نجاح خمس عمليات فقط، وفشل خمس عشرة عملية بسبب عدم كفاية المخزون، وانتهى المخزون عند الصفر دون كمية سالبة ودون بيع زائد.

النتيجة المحفوظة	المقياس
5	المخزون الابتدائي
20	المستخدمون المتزامنون
5	عمليات الإتمام الناجحة
15	عمليات الإتمام المرفوضة
0	المخزون النهائي
لا يوجد	بيع زائد أو مخزون سالب

مسار الملف في المشروع

orders/views.py | lines 81-101

```
with transaction.atomic():
    cart = Cart.objects.select_for_update().get(user=request.user)
    cart_items = list(
        cart.items.select_for_update().order_by("product_id", "id")
    )
```

```

product_ids = sorted({item.product_id for item in cart_items})
locked_products = {
    product.id: product
    for product in Product.objects.select_for_update()
    .filter(id__in=product_ids)
    .order_by("id")
}

for item in cart_items:
    product = locked_products[item.product_id]
    if product.stock < item.quantity:
        return Response({"error": "Out of stock"}, status=400)

```

ملف الإثبات: scripts/race_condition_test.py

النتيجة المحفوظة: results/race_condition/race_condition_task1_latest.json

المتطلب الثاني: إدارة الموارد والتحكم بعدد عمليات الإتمام 5.

حتى لو كانت عملية الإتمام آمنة من ناحية سلامة المخزون، فإن السماح بعدد غير محدود من عمليات الإتمام المتوازية قد يستهلك اتصالات قاعدة البيانات والخيوط والذاكرة، فيتحول الضغط إلى بطء شديد أو انهيار. لذلك يضع المشروع سقفاً لعدد عمليات الإتمام النشطة في الوقت نفسه.

التنفيذ لا يعتمد على عداد داخل نسخة واحدة من التطبيق، لأن المشروع يعمل على أكثر من نسخة بدلاً من ذلك يستخدم عداداً ذرياً تزيد كل عملية العداد عند الدخول، ويُرفض الطلب إذا تجاوز الحد، ثم يُنقص العداد عند الخروج. بهذه الطريقة Redis مشتركاً داخل. تتشارك نسخ التطبيق الثلاث نفس عدد العمليات النشطة

مسار الملف في المشروع

performance/capacity_limiter.py | lines 58-72

```

self.active_count = int(self.redis_client.incr(self.key))
self.redis_client.expire(self.key, self.ttl_seconds)

if self.active_count > self.limit:
    self.redis_client.decr(self.key)
    self.acquired = False
    return False

self.acquired = True
update_max_observed(self.redis_client, self.max_key, self.active_count)
return True

```

نتيجة الإثبات

الاختبار المحفوظ ضبط الحد عند ست عمليات إتمام ونفذ عشرين مستخدماً متزامناً. سجلت النتيجة ثماني عمليات ناجحة واثنى عشرة عملية رفضت بسبب السعة، وكان أكبر عدد مرصود للعمليات النشطة ستاً تماماً، أي إن الحد احترم ولم يحدث خطأ خادمي.

ملف الإثبات: scripts/resource_capacity_test.py

النتيجة المحفوظة: results/resource_capacity/resource_capacity_task2_latest.json

المتطلب الثالث: المعالجة غير المتزامنة 6.

لا يحتاج المستخدم إلى انتظار إنشاء فاتورة أو إرسال إشعار كي يستلم نتيجة عملية الشراء. إذا نُفذت هذه الأعمال ضمن مسار الطلب نفسه فإن زمن الاستجابة يتضاعف، ويصبح خادم الويب مشغولاً بأعمال يمكن نقلها إلى عامل خلفي في المشروع، لا ترسل المهام الخلفية قبل اعتماد معاملة الشراء. يسجل النظام أولاً حالة المهمة في جدول مخصص، ثم يرسلها إلى Celery بعد نجاح الاعتماد. بهذا لا يمكن للعامل الخلفي أن يرى طلباً تم التراجع عنه لاحقاً Celery

مسار الملف في المشروع

orders/views.py | lines 138-142

```
transaction.on_commit(
    lambda product_ids=changed_product_ids:
        invalidate_checkout_related_caches(product_ids)
)
transaction.on_commit(lambda: dispatch_order_tasks(order.id))
```

مسار الملف في المشروع

orders/views.py | lines 29-47

```
task_specs = [
    ("generate_invoice_task", generate_invoice_task),
    ("send_order_notification_task", send_order_notification_task),
]

for task_name, task_func in task_specs:
    task_log = OrderBackgroundTask.objects.create(
        order_id=order_id,
        task_name=task_name,
        status=OrderBackgroundTask.Status.QUEUED,
    )
    async_result = task_func.delay(
        order_id,
        background_task_id=task_log.id,
    )
```

النتيجة المحفوظة أثبتت أن طلب الإتمام عاد برمز نجاح قبل اكتمال المهمتين الخلفيتين. تم إنشاء مهمتين، وانتهتا بنجاح، وبلغ مجموع زمن العمل الخلفي نحو 2054 ميلي ثانية بينما استجاب طلب الإتمام خلال نحو 77.18 ميلي ثانية في النتيجة الأخيرة.

ملف الإثبات: scripts/async_queue_test.py

النتيجة المحفوظة: results/async_queues/async_queue_task3_latest.json

المتطلب الرابع: المعالجة الدفعية لتقارير المبيعات 7.

التقرير اليومي قد يضم عدداً كبيراً من الطلبات. تحميل جميع السجلات ثم تجميعها دفعة واحدة يستهلك الذاكرة ويجعل زمن التنفيذ غير قابل للتوقع. لذلك يقسم المشروع العمل إلى دفعات محدودة الحجم. تعالج كل دفعة جزءاً من الطلبات، ثم تحفظ مؤشرات التقدم والمجاميع المرحلية.

كما تحصل قبل التنفيذ على قفل موزع لليوم المطلوب كي لا يبدأ عاملان بحساب التقرير، Celery عملية التقرير نفسها تعمل في عامل لنفس التاريخ.

مسار الملف في المشروع

reports/tasks.py | lines 73-91

الصفحة 7

```

with redis_distributed_lock(
    daily_sales_report_lock_key(report_date),
    timeout=getattr(
        settings,
        "DAILY_SALES_DISTRIBUTED_LOCK_TIMEOUT_SECONDS",
        300,
    ),
    blocking_timeout=0,
) as report_lock:
    if not report_lock.acquired and report_lock.status == LOCK_BUSY:
        return mark_daily_sales_batch_run_skipped(
            report_date=report_date,
            batch_run_id=batch_run_id,
            chunk_size=chunk_size,
            reason="distributed_lock_busy",
        )

    return run_daily_sales_report_processing(
        self, report_date, batch_run_id, chunk_size
    )

```

الاختبار المحفوظ أنشأ 250 طلباً تجريبياً بحجم دفعة يساوي 50. النتيجة كانت خمس دفعات بالضبط، مع تطابق إجمالي الكميات والمبيعات مع القيم المتوقعة، وتسجيل نجاح عملية التقرير.

النتيجة	المقياس
250	الطلبات التجريبية
50	حجم الدفعة
5	عدد الدفعات المتوقع والفعلي
750	إجمالي الكمية المباعة
11355.00	إجمالي المبيعات
نجاح	حالة العملية

ملف الإثبات: scripts/batch_processing_test.py

النتيجة المحفوظة: results/batch_processing/batch_processing_task4_latest.json

المتطلب الخامس: توزيع الأحمال 8.

وجود قفل صحيح ومعاملة صحيحة لا يعني أن خادماً واحداً يكفي. لذلك يشغل المشروع ثلاث نسخ من تطبيق الويب ويضع أمامها موزع كمصادر حالة مشتركة Redis أحمال. هذا يسمح بتوزيع طلبات المستخدمين على نسخ متعددة مع بقاء قاعدة البيانات و

اختيرت استراتيجية التوزيع الدوري، أي إن الطلبات المتتالية توزع بالتناوب بين النسخ المتاحة. كما يملك موزع الأحمال فحص صحة دوري لمسار الصحة، فلا يرسل طلبات إلى نسخة غير جاهزة

مسار الملف في المشروع

infra/haproxy/haproxy.cfg | lines 18-24

```

backend django_backends
    balance roundrobin
    option httpchk GET /api/health/

```



```
http-check expect status 200
server web1 web:8000 check inter 2s fall 3 rise 2
server web2 web2:8000 check inter 2s fall 3 rise 2
server web3 web3:8000 check inter 2s fall 3 rise 2
```

نتيجة الاختبار المحفوظة أرسلت 61 طلباً ناجحاً. وصلت 21 طلباً إلى النسخة الأولى و20 إلى الثانية و20 إلى الثالثة، وهذا توزيع متوازن عملياً ويثبت أن النسخ الثلاث تشارك فعلاً في تخدم الطلبات.

ملف الإثبات: scripts/load_distribution_test.py

النتيجة المحفوظة: results/load_distribution/load_distribution_task5_latest.json

المتطلب السادس: التخزين المؤقت في أكثر من موضع. 9.

ضمن المشروع؛ المطلوب هو استخدامه في أماكن قراءة متكررة لتقليل الاستعلامات المباشرة على Redis لا يكفي وجود طبق المشروع التخزين المؤقت لقائمة المنتجات وتفاصيل المنتج، وكذلك لقائمة تقارير المبيعات وتفاصيل تشغيل PostgreSQL. التقرير. أي أن التخزين المؤقت موزع على مجالين مختلفين: المنتجات والتقارير. كل استجابة مخزنة تشير صراحة إلى ما إذا كانت أعيدت من الذاكرة أو بنيت من المصدر. يُرسل المشروع رأساً يوضح حالة التخزين المؤقت، وبذلك يمكن إثبات سلوك التخزين دون الاعتماد على فرق زمني قد يتأثر بالجهاز.

مسار الملف في المشروع

products/views.py | lines 46-66

```
cached_data = cache.get(PRODUCT_LIST_CACHE_KEY)
if cached_data is not None:
    return set_cache_header(Response(cached_data), CACHE_HIT)

with redis_distributed_lock(
    product_list_cache_rebuild_lock_key(),
    timeout=10,
    blocking_timeout=2,
) as rebuild_lock:
    cached_data = cache.get(PRODUCT_LIST_CACHE_KEY)
    if cached_data is not None:
        return set_cache_and_lock_headers(
            Response(cached_data),
            CACHE_HIT,
            lock_status,
        )

    response = super().list(request, *args, **kwargs)
    cache.set(
        PRODUCT_LIST_CACHE_KEY,
        response.data,
        timeout=PRODUCT_LIST_CACHE_TIMEOUT_SECONDS,
    )
    return set_cache_and_lock_headers(
        response, CACHE_MISS, rebuild_lock.status
    )
```

مسار الملف في المشروع

products/cache_utils.py | lines 8-18

```
PRODUCT_LIST_CACHE_KEY = "products:list:v1"
```

```

PRODUCT_DETAIL_CACHE_KEY_PREFIX = "products:detail:v1"
DAILY_SALES_REPORTS_LIST_CACHE_KEY = "reports:daily-sales:list:v1"
DAILY_SALES_BATCH_RUN_DETAIL_CACHE_KEY_PREFIX = (
    "reports:daily-sales:batch-run:v1"
)

PRODUCT_LIST_CACHE_TIMEOUT_SECONDS = 60
PRODUCT_DETAIL_CACHE_TIMEOUT_SECONDS = 120
DAILY_SALES_REPORTS_LIST_CACHE_TIMEOUT_SECONDS = 60
DAILY_SALES_BATCH_RUN_DETAIL_CACHE_TIMEOUT_SECONDS = 30

```

يُضاف إلى ذلك إبطال منظم للتخزين المؤقت بعد اعتماد عملية الشراء، لأن عملية الشراء تغير المخزون وتؤثر في بيانات المبيعات .
وُضع الإبطال ضمن عملية تنفذ بعد الاعتماد حتى لا تحذف بيانات الذاكرة بسبب طلب فشل وتراجع.

مسار الملف في المشروع

products/cache_utils.py | lines 65-67

```

def invalidate_checkout_related_caches(product_ids):
    invalidate_product_caches(product_ids)
    invalidate_daily_sales_report_caches()

```

نتيجة اختبار التخزين المؤقت تحققت من أربعة مسارات. سجلت كل المسارات انتقالاً من حالة عدم وجود بيانات مخزنة إلى حالة استخدام بيانات مخزنة. كما تحسن أو لم يسؤ زمن الطلب الثاني في ثلاث من أربع حالات، مع التأكيد أن رأس الاستجابة هو الدليل الأساسي على صحة السلوك.

ملف الإثبات: scripts/cache_test.py

النتيجة المحفوظة: results/cache/cache_task6_latest.json

المتطلب السابع: الأقفال الموزعة خارج قاعدة البيانات. 10

قفل صفوف قاعدة البيانات ضروري لحماية المخزون نفسه، لكنه لا يغطي كل أشكال التزامن في نظام موزع. هناك حالات يجب ضبطها قبل دخول قاعدة البيانات أو خارجها، مثل إرسال طلب إتمام مكرر لنفس المستخدم، أو تشغيل التقرير اليومي نفسه من عاملين، أو محاولة عدة نسخ إعادة بناء نفس بيانات التخزين المؤقت بعد انتهاء صلاحيتها.

لهذا يوجد في المشروع مكوّن عام للقفل الموزع. يضع كل قفل ضمن مساحة أسماء موحدة ويعيد حالة مفهومة: يتم الحصول على القفل، أو تم الانتظار، أو القفل مشغول. ولا تُنفذ عملية إعادة بناء بيانات التخزين المؤقت إذا بقي القفل مشغولاً.

مسار الملف في المشروع

performance/distributed_locks.py | lines 36-48

```

def checkout_user_lock_key(user_id):
    return normalize_lock_key(f"checkout:user:{user_id}")

def daily_sales_report_lock_key(report_date):
    return normalize_lock_key(f"daily-sales:{report_date}")

def product_list_cache_rebuild_lock_key():
    return normalize_lock_key("cache-rebuild:products:list")

```

مسار الملف في المشروع

orders/views.py | lines 64-79

```
with redis_distributed_lock(
    checkout_user_lock_key(request.user.id),
    timeout=15,
    blocking_timeout=0,
) as checkout_lock:
    if (
        not checkout_lock.acquired
        and checkout_lock.status == LOCK_BUSY
    ):
        return Response(
            {
                "code": "checkout_lock_busy",
                "detail": "Checkout is already running for this user.",
            },
            status=status.HTTP_429_TOO_MANY_REQUESTS,
        )

    return self.run_checkout(request)
```

اختبار الأقفال الموزعة نفذ اثنتي عشرة محاولة. حصلت ثلاث محاولات فقط على الأقفال بينما حجت تسع محاولات، كما أثبت أن القفل المكرر لعملية الإتمام يعيد حالة الانشغال، وأن تشغيل تقرير اليوم نفسه أثناء وجود قفل له يتجاوز التنفيذ بدلاً من تشغيله مرتين.

ملف الإثبات: scripts/distributed_lock_test.py

النتيجة المحفوظة: results/locks/task7_distributed_lock_latest.json

المتطلب الثامن: سلامة المعاملات 11.

عملية الشراء ليست محدثاً واحداً، بل سلسلة مترابطة من العمليات: إنشاء الطلب، إنشاء عناصره، تخفيض المخزون، إنشاء الدفع، وحذف عناصر السلة. لو نجحت بعض الخطوات وفشلت أخرى فسيبقى النظام في حالة خاطئة، مثل دفع بلا طلب أو مخزون منخفض بلا سجل بيع.

يعالج المشروع هذا الأمر بوضع كل التغييرات الدائمة ضمن معاملة واحدة. إذا حدث استثناء قبل الاعتماد، تتراجع جميع الكتابات التي حدثت داخل المعاملة. أما الأعمال الخارجية مثل إبطال التخزين المؤقت وإرسال مهام الخلفية فتسجل لتعمل فقط بعد الاعتماد الناجح.

مسار الملف في المشروع

orders/views.py | lines 103-142

```
with transaction.atomic():
    order = Order.objects.create(
        user=request.user,
        total_price=total_price,
        status=Order.Status.PAID,
    )

    product.stock -= item.quantity
    product.save(update_fields=["stock"])

    OrderItem.objects.bulk_create(order_items)

    Payment.objects.create(
        order=order,
        amount=total_price,
        status=Payment.Status.COMPLETED,
        transaction_reference=f"TXN-{uuid4().hex[:20].upper()}",
    )
```

```
)
cart.items.all().delete()

transaction.on_commit(
    lambda product_ids=changed_product_ids:
        invalidate_checkout_related_caches(product_ids)
)
transaction.on_commit(lambda: dispatch_order_tasks(order.id))
```

اختبار سلامة المعاملة يتضمن حالتين. في الحالة الناجحة عاد الطلب برمز 201، نقص المخزون، حذفت السلة، وأنشئ الطلب وعناصره والدفع. وفي حالة فشل مقصودة بعد تخفيض المخزون، ثبت أن المخزون عاد كما كان، وبقيت السلة، ولم ينشأ طلب أو دفع أو مهمة خلفية. هذا اختبار عملي مباشر لفكرة أن العملية تنجح كاملة أو تفشل كاملة.

ملف الإثبات: scripts/acid_transaction_test.py

النتيجة المحفوظة: results/acid/task8_acid_transaction_latest.json

المتطلب التاسع: محاكاة مئة مستخدم على العمليات الفعلية. 12.

المطلوب ليس إرسال مئة طلب بسيط إلى قائمة المنتجات، بل محاكاة تدفق المستخدم الحقيقي على العمليات الموجودة في النظام. لهذا ينشئ سكربت الاختبار مئة مستخدم منفصل، ويمنح كل مستخدم رمز وصول خاصاً، ثم يشغل العمليات المتسلسلة الخاصة به بالتوازي مع بقية المستخدمين.

كل مستخدم في الاختبار يقرأ قائمة المنتجات، يقرأ تفاصيل منتج، يضيف المنتج إلى السلة، يعدل كمية العنصر، ينفذ الإتمام، يقرأ سجل طلباته، ثم يقرأ تفاصيل المنتج مرة ثانية لاختبار المسار المخزن، ويستعلم عن قياس السعة وعن تقرير المبيعات. كما ينشئ مستخدم مدير مستقل لاختبار مسارات التقرير التي تحتاج صلاحية أعلى.

مسار الملف في المشروع

scripts/stress_test_100_users.py | lines 312-334

```
flow = [
    (
        "get_product_list",
        lambda: get_json(base_url, "/api/products/", token=token),
        SUCCESS_STATUS_CODES,
    ),
    (
        "get_product_detail",
        lambda: get_json(
            base_url,
            f"/api/products/{product_id}/",
            token=token,
        ),
        SUCCESS_STATUS_CODES,
    ),
    (
        "add_product_to_cart",
        lambda: post_json(
            base_url,
            "/api/cart/items/",
            {"product": product_id, "quantity": quantity},
            token=token,
        ),
        {201},
    ),
]
```

```
),  
]
```

مسار الملف في المشروع

scripts/stress_test_100_users.py | lines 361-383

```
checkout_record = timed_operation(  
    "checkout",  
    lambda: post_json(  
        base_url,  
        "/api/orders/checkout/",  
        {},  
        token=token,  
        extra_headers=checkout_headers,  
    ),  
)  
  
orders_record = timed_operation(  
    "get_order_history",  
    lambda: get_json(base_url, "/api/orders/", token=token),  
)
```

بعد انتهاء التشغيل، لا يكفي السكربت بحساب عدد الردود؛ بل يراجع حالة البيانات داخل قاعدة البيانات: لا مخزون سالب، لا بيع أكبر من المخزون، المخزون النهائي يساوي المخزون الابتدائي ناقص الكمية المباعة، وعدد الطلبات المدفوعة وعدد الدفعات يطابقان عدد عمليات الإتمام الناجحة.

النتيجة المحفوظة	المقياس
100	المستخدمون المحاكون
1003	إجمالي العمليات والطلبات المسجلة
100	المستخدمون الناجحون
100	عمليات الإتمام الناجحة
0	الأخطاء أو فشل الطلبات
151.9	معدل الطلبات في الثانية
619.01 ميلي ثانية	متوسط زمن الاستجابة
1905.39 ميلي ثانية	الزمن عند الشريحة 95 بالمئة

ملف الإثبات: scripts/stress_test_100_users.py

النتيجة المحفوظة: results/stress/task9_stress_100_users_latest.json

المتطلب العاشر: القياس وتحليل الاختناقات 13.

يحتاج تحليل الأداء إلى قياس وليس انطباعاً. لهذا يغلف المشروع جميع الطلبات بمكون وسيط يقيس الزمن منذ دخول الطلب حتى إنتاج الاستجابة، ثم يسجل المسار والطريقة ورمز الحالة والزمن داخل جدول سجلات الأداء. هذا تطبيق عملي لفكرة مراقبة الأداء كاهتمام عام يقطع جميع أجزاء التطبيق، بدلاً من نسخ كود القياس داخل كل واجهة.

```
class PerformanceLogMiddleware:
    SKIP_PATHS = {"/api/health/", "/api/server-info/"}

    def __call__(self, request):
        start = time.perf_counter()
        response = self.get_response(request)
        duration_ms = (time.perf_counter() - start) * 1000

        PerformanceLog.objects.create(
            endpoint=request.path[:255],
            method=request.method,
            status_code=response.status_code,
            duration_ms=duration_ms,
        )
        return response
```

أما سكربت المئة مستخدم فيجمع المتوسط والحد الأدنى والحد الأقصى والزمن عند الشريحة 95 بالمئة لكل عملية، إضافة إلى معدل الطلبات في الثانية ومعدل الخطأ. ومن نتائج التشغيل المحفوظة يظهر أن عملية الإتمام هي من أكثر العمليات كلفة تحت الضغط: متوسطها نحو 1000.44 ميلي ثانية، وزمن الشريحة 95 بالمئة نحو 2263.77 ميلي ثانية. هذه النتيجة منطقية لأن الإتمام يجمع قفل السعة والقفل الموزع وأقفال الصفوف والكتابة في عدة جداول.

عدد الأخطاء	زمن الشريحة 95 بالمئة	المتوسط	العملية
0	ميلي ثانية 2284.11	ميلي ثانية 880.81	قراءة قائمة المنتجات
0	ميلي ثانية 2008.44	ميلي ثانية 729.12	إضافة منتج إلى السلة
0	ميلي ثانية 1959.41	ميلي ثانية 696.92	تعديل عنصر السلة
0	ميلي ثانية 2263.77	ميلي ثانية 1000.44	إتمام الطلب
0	ميلي ثانية 1682.88	ميلي ثانية 570.21	قراءة سجل الطلبات

الاختناق الذي يمكن إثباته من هذه البنية هو مسار الإتمام، لا لأنه يحتوي خطأ، بل لأنه أكثر مسار يطلب تماسكاً: يقوم بالتحقق والكتابة والقفل وإرسال آثار لاحقة. التحسينات المطبقة بالفعل لتخفيف الضغط هي التخزين المؤقت لمسارات القراءة، نقل الفواتير والإشعارات إلى الخلفية، وتوزيع الدخول على ثلاث نسخ. لكن المقارنة الرقمية الرسمية قبل التحسين وبعده لعملية محددة لا تظهر كملف مستقل في النتائج الحالية؛ لذلك يجب عدم الادعاء بأنها مكتملة إن سأل الدكتور تحديداً عن المقارنة قبل وبعد.

النقطة الناقصة بوضوح في المتطلب العاشر: القياس وتحديد الاختناق موجودان فعلاً، لكن لإنهاء المتطلب بصيغته الحرفية يجب تشغيل السيناريو نفسه مرتين على الأقل: مرة مع تعطيل التخزين المؤقت لمسار قراءة محدد، ومرة بعد تفعيله، ثم وضع المتوسط وزمن الشريحة 95 بالمئة ومعدل الطلبات في الثانية في جدول مقارنة واحد.

ملفات القياس: performance/middleware.py + scripts/stress_test_100_users.py

ملف نتائج الأداء: results/stress/task9_stress_100_users_summary.md

نتائج الاختبارات المحفوظة وكيفية عرض المشروع 14.

المتطلب	الأمر المقترح داخل الحاوية	ما يجب أن يظهر
تضارب المخزون	python scripts/race_condition_test.py	نجاح الاختبار وعدم البيع الزائد
ضبط السعة	python scripts/resource_capacity_test.py	احترام الحد ورفض الطلبات الزائدة
المهام الخلفية	python scripts/async_queue_test.py	إتمام الشراء قبل انتهاء المهمة
المعالجة الدفعية	python scripts/batch_processing_test.py	عدد الدفعات يطابق المتوقع
توزيع الأحمال	python scripts/load_distribution_test.py	وصول الطلبات إلى النسخ الثلاث
التخزين المؤقت	python scripts/cache_test.py	HIT إلى MISS انتقال من
الأقفال الموزعة	python scripts/distributed_lock_test.py	بعض المحاولات محجوبة
سلامة المعاملة	python scripts/acid_transaction_test.py	حالة نجاح وحالة تراجع
مئة مستخدم	python scripts/stress_test_100_users.py - -base-url http://load_balancer -- users 100	مستخدم ناجحون دون بيع زائد 100